

Intro to Python

EEC 3150

Outline

- 1 Programming in General
- 2 About Python
- 3 Objects and Operators
- 4 Control Structures: Conditionals and Loops

Programming in General

Defn: “computer programming”

- **Computer programming** is the process of performing a given computational task by designing, building, and executing a computer program (Wikipedia).
- **Computer programs** are sequences of instructions (or commands, statements) that is written in a programming language and designed to complete some computational task
- A **programming language** is the language in which a computer program is written. It contains all of the common features of human languages: vocabulary (set of words), syntax/grammar (rules for constructing sequences of words), semantics (meaning).

Programming in General

Types of programming languages:

- Procedural languages
- Object-oriented languages (OOP)
- Functional languages
- Visual languages
- Assembly languages
- Machine languages
- Hardware description languages
- Query languages
- Synchronous/Asynchronous languages
- Markup/typesetting languages

Python is procedural, object-oriented, and synchronous

Programming in General

Defn: “low-level” vs. “high-level”

Programming languages can be classified into several different categories:

- **Low-level** languages deal with instructions and data objects which are at or near the level of machine code, i.e., individual bits.
- **High-level** languages deal with instructions and data objects which are at a more abstract, human-readable level.

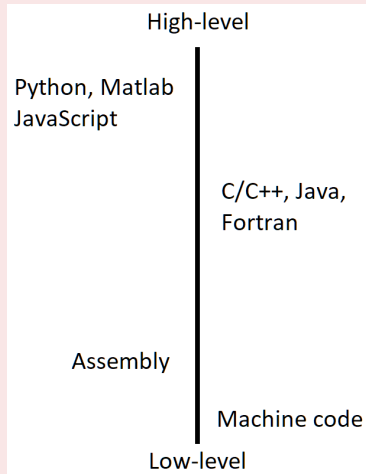


Figure: Levels of programming languages

Defn: “low-level” vs. “high-level”

Low-level:

- Pros: execute faster, more control over under-the-hood aspects
- Cons: code is very precise but verbose/tedious, not as easily readable

High-level:

- Pros: easier to write, complicated tasks can be written in fewer lines of code, under-the-hood stuff handled for you
- Cons: execute slower, less control

Rule of thumb: execution speed and ease of reading/writing are typically inversely related

Defn: “interpreted” vs. “compiled”

To run/execute your program, the computer must convert the code into low-level format that it understands:

- **Interpreted** languages (Python, Matlab, JavaScript) perform this conversion *line-by-line*, as well as execute the program line-by-line.
- **Compiled** languages (C/C++, Java, Fortran) convert (i.e., compile) the *entire* program at once, creating a new file that can be executed.

Pros and Cons :

- Compiled languages are MUCH faster, but must be recompiled whenever a change is made to the source code. The compiled program often cannot be run on another machine.
- Interpreted languages are usually high-level, making them easier to read/write. Also, their error messages can be easier to understand due to the line-by-line nature of the interpreter.

About Python

About Python

Counting backward from 10 in four programming languages:

Do you find anything noteworthy about the Python code?

C

```
#include <stdio.h>
main() {
    int k;
    for (k=10; k>=1; k--) {
        printf("%d\n",k);
    }
}
```

C++

```
using namespace std;
#include <iostream>
main() {
    int k;
    for( k=10; k>=1; k-- ) {
        cout << k << endl;
    }
}
```

Java

```
public class rev1 {
    public static void main(String [] args) {
        int k;
        for( k=10; k>=1; k-- ) {
            System.out.println(k);
        }
    }
}
```

Python

```
for k in range(10,0,-1):
    print k
```

About Python

Counting backward from 10 in four programming languages:

Python:

- Shorter than all the others
- No semicolons
- No curly braces (uses tabs instead)
- No need to declare variables before use

C

```
#include <stdio.h>
main() {
    int k;
    for (k=10; k>=1; k--) {
        printf("%d\n",k);
    }
}
```

C++

```
using namespace std;
#include <iostream>
main() {
    int k;
    for( k=10; k>=1; k-- ) {
        cout << k << endl;
    }
}
```

Python

```
for k in range(10,0,-1):
    print k
```

Java

```
public class rev1 {
    public static void main(String [] args) {
        int k;
        for( k=10; k>=1; k-- ) {
            System.out.println(k);
        }
    }
}
```

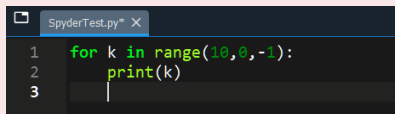
Moral of the story:

Python is very easy to write and you can write in a single line what would take paragraphs in other languages

About Python

Some Python basics:

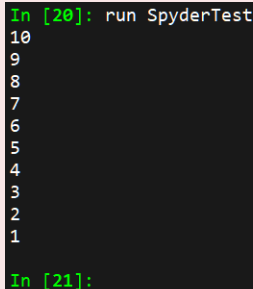
Python programs are called **scripts** (standard convention for interpreted languages)

A screenshot of a code editor window titled 'SpyderTest.py* X'. The code contains a for loop that prints numbers from 10 down to 1. The line numbers 1, 2, and 3 are visible on the left margin.

```
1 for k in range(10,0,-1):  
2     print(k)  
3
```

Figure: Example script from Spyder

Python scripts are executed in a **shell** (command line interface, console, terminal)

A screenshot of a console window showing the execution of the script. The prompt 'In [20]:' is followed by the command 'run SpyderTest'. The output shows the numbers 10 through 1 on separate lines. The next prompt 'In [21]:' is visible at the bottom.

```
In [20]: run SpyderTest  
10  
9  
8  
7  
6  
5  
4  
3  
2  
1  
  
In [21]:
```

Figure: Example console from Spyder

Anaconda3 and Spyder:

- In order to write a Python script, you need either a simple text editor or a more robust **IDE** (*integrated development environment*). IDE's provide many additional features that simple text editors lack (debugging, syntax highlighting, auto-completion, profiling, etc.).
- In order to run a Python script, you need to have some version of Python installed on your machine

We will use Anaconda3, an open-source distribution of Python featuring most major scientific libraries and the Spyder IDE



Getting started with Spyder:

- Install and launch Anaconda3 ([LINK](#)), Open Spyder
- Examine different panes:
Editor, console, files, etc.
- Explore color themes:
Tools > Preferences > Appearance
- Disable auto-completion features:
Tools > Preferences > Editor > Source Code
Tools > Preferences > Completion and Linting
- Explore panes and layouts (under View tab)

Getting started with Spyder:

- Write and run (F5) simple script:
`print('Hello World')`
- Use the file pane to navigate to an appropriate folder to save this script as `HelloWorld.py`
- While in that folder, you can also run the script by typing `run HelloWorld.py` in the console and pressing Enter

Objects and Operators

Objects and Operators

Intro: objects and operators

Two of the main entities used in Python are

- **Objects:** pieces of data
Ex: numbers, strings, lists, functions
- **Operators:** things which perform operations on objects
Ex: number addition, string concatenation, comparison

Data types:

Objects can be one of several fundamental data types (there are additional derived data types):

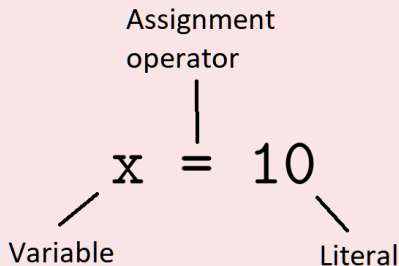
- **Integers** (`int`)
Ex: 0, 1, -145350, 2^{20}
- **Floating-point numbers** (`float`)
Ex: 0.0, 1.0, 3.14159265, -2.4e4
- **Booleans** (`bool`)
Ex: True, False
- **Strings** (`str`)
Ex: 'a', 'qwerty', 'python is great'

(Note: you can use either single or double quotes for strings, just make sure both sides match)

Objects and Operators

Literals and variables:

- **Literals** are constant values of some data type. They do not change and are not stored in memory.
Ex: 100, 3.14e10, 'KD9-3.7'
- **Variables** are labeled locations in memory which can store data
Ex: x, count, filename
- To *define* a variable, use the **assignment operator**: =
- You can change the value of variables
- You can even change their type
- You can't use a variable prior to defining it



Objects and Operators

Notes on whitespace:

- `x=10`, `x = 10`, `x     =10` are all interpreted the same way in Python
- You can also have as many blank lines as you want between non-blank lines
- Indentation is the only whitespace that matters (more on that later)

Rules for variable names:

- Variables *can* have upper/lower case characters, numbers, underscores
- Variables *can't* have other symbols or start with numbers
- Variables *can't* be pre-existing keywords: `for`, `in`, `while`, `if`
- A common convention for variable naming is *camel case*:
`isGreaterThanZero`, `stockPrice`
- Valid variable names: `_x`, `test00`, `isNew`, `is_new`, `zz45Z35ZZz`
- Invalid variable names: `is new`, `0test`, `length&OfString`

Objects and Operators

Keywords:

Python also has a list of reserved keywords which CANNOT be used as variables:

and	break	elif	for	in	not	True
as	class	else	from	is	or	try
assert	continue	except	global	lambda	pass	while
async	def	False	if	nonlocal	raise	with
await	del	finally	import	None	return	yield

Multiple assignment:

- Python allows you to assign multiple variables in one line:
Ex: `x, y = 1, 2`
- Useful when dealing with functions that return multiple outputs

Objects and Operators

Operators:

Operators take one or more objects and return new ones

Examples:

- Numeric: `+`, `-`, `*`, `/`, `**` (exponent), `%` (remainder)
Note 1: numeric operators follow PEMDAS, group with parentheses
Note 2: try mixing floats and ints in the same operation
- Boolean: `==` (comparison, checks for equality), `!=` (not equal), `<`, `>`, `<=`, `>=`, `not`, `and`, `or`
- String: `+` (concatenation), `*` (repetition), `[<int>]` (slicing/indexing, starts at 0, substring with colon), `in` (membership)

Bonus: comments

Use the `#` character to comment out a line

Expressions:

Expressions are formulas of a certain data type (the same as in math)

Examples:

- Numeric expressions return numeric values as their output

$(1.0 + 3 * 5) ** 0.5$

$b * h / 2$

- Boolean expressions return True/False as their output

$1 > 2$

$x == 1$ and $y == 2$

- String expressions return strings as their output

`'a' + 'b'`

`('a' + 'b') * 2`

Objects and Operators

Super brief intro to functions:

- A **function** is a piece of code which takes some number of arguments as input and produces some sort of output
- Terminology:
An argument is **passed** to a function
The function **returns** the output
(Note: some functions don't return anything, some don't need arguments)
- Examples of functions:
`print()` (displays text on console, does not return anything)
`input()` (reads keyboard input from the console)
`abs()` (returns absolute value of numeric input)
`max()` (takes several numeric arguments, returns the maximum)
- Let's try passing different types to these functions

Objects and Operators

Console/shell prompt notation:

- The notation `>>>` denotes typing and running a command directly into a Python console
- The output of the command will be placed on the line below it without the notation

The `type()` function:

Use the Python function `type()` to discover what data type its input is:

- ```
>>> type(100)
int
```
- ```
>>> type("red")
str
```

NOTE: unless typing directly in the console, you must `print()` the output in order to see it in the console: `print(type(100))`

Objects and Operators

Casting data:

Casting is changing some data object from one type to another

- To cast, simply use the abbreviated name of the desired type as a function and pass your data to it
- Ex: `str(100)` will convert the integer 100 to the string '100'
- Ex: `int("100")` will convert the string '100' to the integer 100
- Ex: `int("abc")` will *throw an error*

Casting floats and ints:

- When casting a float to an int, it always rounds down:

```
>>> int(1.7)
1
```
- When casting an int to a float, it appends a .0:

```
>>> float(1)
1.0
```

Objects and Operators

Weirdness with ints and floats:

Arguments to functions:

- Be aware that some functions take ints as arguments, so passing floats will throw an error
- Passing an int to a function which takes floats is fine since it can simply cast it

Integer division:

- In many languages (as well as previous versions of Python), the division of two integers would produce a truncated integer rather than a float
- As of Python 3 (the version we're using), integer division (`//`) and float division (`/`) have their own operators
- Keep in mind whether you need an integer or a float for specific applications (like string indexing)

Control Structures: Conditionals and Loops

Preview: conditionals and loops

Two critical programming constructs which control the flow of a program are:

- **Conditionals** (i.e., if statements)
These cause the program to branch to one of several blocks of code based on whether different logical conditions are met
- **Loops** (i.e., for loops and while loops)
These cause the program to repeat blocks of code until a certain condition is met

Conditionals and Loops

Conditionals (if statements):

Prototypical if statement:

```
if boolean expression:  
    block of code  
else:  
    block of code
```

(Note: italics in code snippets mean "some code goes here")

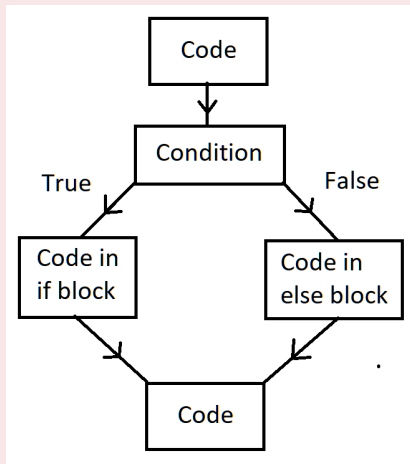


Figure: Flowchart for an if-else statement

Conditionals and Loops

Conditionals (if-else statements):

```
x = 11
```

```
if x%2 == 0:  
    print("x is even")  
else:  
    print("x is odd")
```

```
print("Done")
```

Indentation:

- Unlike most languages, **indentation is meaningful in Python**
- It is used to specify blocks of all kinds (if statements, loops, functions, classes, etc.)

- The Boolean expression should return either True or False
- Terminate the if/else lines with a colon (this signifies the start of a new code block)
- Indent all lines which should be part of each case's block
- To finish, revert one level of indenting

Conditionals and Loops

3 kinds of if statements:

You can have 1, 2, or more cases:

		<code>if bool exp:</code>	
		<code> block of code</code>	
<code>if bool exp:</code>	<code> block of code</code>	<code>elif bool exp:</code>	
<code> block of code</code>	<code>else:</code>	<code> block of code</code>	
	<code> block of code</code>	<code>else:</code>	
		<code> block of code</code>	

- `elif` is short for "else if"
- You can have as many `elif` case as you want
- You can have `elif` without the `else`
- Once the interpreter finds the correct case, the rest are skipped

Conditionals and Loops

Nested if's:

```
x = 11

if x%2 == 0:
    if x%3 == 0:
        print("x is divisible by 2 and 3")
    else:
        print("x is divisible by 2, not 3")
elif x%3 == 0:
    print("x is divisible by 3, not 2")
else:
    print("x isn't divisible by 2 or 3")

print("Done")
```

Conditionals and Loops

Don't use separate if statements to account for all cases:

The code on the LEFT is better:

```
if x%2 == 0:
    print("x is even")
else:
    print("x is odd")
```

```
if x%2 == 0:
    print("x is even")
if x%2 == 1:
    print("x is odd")
```

Two issues:

- It evaluates an unnecessary Boolean expression, wasting computing time (though not much)
- If the conditions were more complex, it is possible it might miss additional cases (consider $x\%3$)

Conditionals and Loops

Using `#end` comments:

- Many languages require a keyword like `end` to finish if statements
- Python doesn't due to indentation, but adding them via comments can be a helpful visual aid

```
if x%2 == 0:
    if x%3 == 0:
        print("x is divisible by 2 and 3")
    else:
        print("x is divisible by 2, not 3")
    # end
elif x%3 == 0:
    print("x is divisible by 3, not 2")
else:
    print("x isn't divisible by 2 or 3")
# end
```

Conditionals and Loops

Exercise:

Write a Python script that will print the largest of three numbers (x,y,z)

Conditionals and Loops

Loops:

Loops are coding constructs which repeatedly loop over (iterate) blocks of code until some condition is met

Two kinds of loops:

- **For loops** iterate over blocks a specified number of times and have an index variable that iterates through the number of iterations of the loop
- **While loops** iterate over blocks while a certain condition is true and then finish once the condition is false

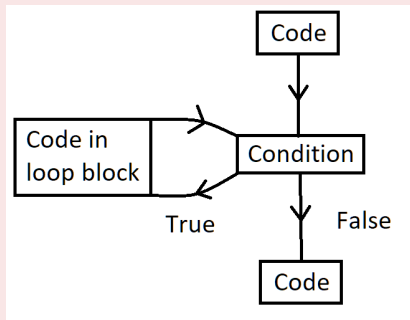


Figure: Flowchart for a while loop

Conditionals and Loops

First for loop:

Here's a for loop which prints the integers from 0-9:

```
for i in range(10):  
    print(i)
```

How does range work:

- Start, end, increment

- Like with if's, use a colon to terminate the first line and indent the body of the block
- `range(10)` is a "list-like" object (it contains the sequence of values 0, 1, ..., 9)
- There are many list-like objects we can iterate over (we will learn about them later)
- For each iteration, `i` is set to a value in `range(10)` (via the keyword `in`) and can then be used in the body of the block

Conditionals and Loops

For loop examples:

Sum the integers 1-10:

```
n = 10
total = 0
for i in range(1,n+1):
    total = total + i
```

Print even integers up to 10 (inclusive):

```
n = 10
for i in range(n+1):
    if i%2 == 0:
        print(i)
```

Conditionals and Loops

First while loop:

Here's a while loop which prints the integers from 0-9:

```
i = 0
while i < 10:
    print(i)
    i += 1
```

- Use a colon and indentation
- The loop keeps iterating until $i < 10$ is no longer True
- $i += 1$ is Python shorthand for writing $i = i + 1$ (you can do this with any operator)
- $i += 1$ increments the counter variable i

Infinite loops:

- Comment out the increment line, code loops forever

Conditionals and Loops

While loop examples:

Print perfect squares less than
or equal to 100:

```
i = 2
while i**2 < 100:
    print(i**2)
    i += 1
```

Continue and break:

- Use `continue` to force a loop to skip the rest of the current iteration and jump to the next
- Use `break` to force a loop to skip the rest of the current iteration and end the loop

Conditionals and Loops

Continue and break:

Print even integers up to 10:

```
i = 0
while True:
    if i%2 != 0:
        i += 1
        continue
    else:
        print(i)
    # end
    if i == 10:
        break
    # end
    i += 1
# end
```

```
for i in range(10+1)::
    if i%2 != 0:
        continue
    else:
        print(i)
    # end
    if i == 10:
        break
    # end
# end
```